

TP7 - DevSecOps : La securite integree au pipeline CI/CD

Objectifs

A la fin de ce TP, vous serez capables de :

- Comprendre le concept DevSecOps et la philosophie "shift left"
- Intégrer une analyse statique de code (SAST) dans un pipeline GitLab CI
- Scanner les dependances pour detecter les vulnerabilites connues
- Detecter les secrets codes en dur (mots de passe, clefs API, tokens)
- Generer un rapport de securite synthetique
- Configurer des seuils de securite via des variables CI/CD

Prerequis

- Avoir realise les TP1 a TP6 (pipelines de base a avances)
- Un compte GitLab avec acces a un Runner
- Connaître les bases de Node.js et npm
- Comprendre les stages, jobs, cache et artifacts GitLab CI

Etape 1 : Qu'est-ce que le DevSecOps ? (theorie)

L'analogie du gardien de securite

Imaginez un immeuble de bureaux :

- **Approche traditionnelle** : Un seul gardien a l'entree principale. Si quelqu'un passe cette porte, il a acces a tout. On ne verifie la securite qu'a la fin, au moment de la mise en production.
- **Approche DevSecOps** : Un gardien a chaque porte, a chaque etage, a chaque salle. La securite est presente partout, a chaque etape. On verifie la securite en continu, des le premier commit.

Le "Shift Left"

Approche traditionnelle :

Code --> Test --> Build --> Deploy --> SECURITE (trop tard !)

Approche DevSecOps :

SECURITE --> Code --> SECURITE --> Test --> SECURITE --> Build --> SECURITE --> Deploy

L'idée est de **deplacer la securite vers la gauche** (vers le debut) du cycle de developpement. Plus un probleme de securite est detecte tot, moins il coute cher a corriger.

Les 3 piliers du DevSecOps dans ce TP

Pilier	Description	Outil dans ce TP
SAST	Analyse statique du code source	Script Node.js personnalis�
Dependency Scan	Scan des dependances npm	<code>npm audit</code>
Secret Detection	Detection de secrets codes en dur	Script Node.js personnalis�

Etape 2 : Creer le projet sur GitLab

1. Connectez-vous a GitLab
2. Cliquez sur **New project** > **Create blank project**
3. Nom du projet : `tp7-devsecops`
4. Visibilite : **Private**
5. Cochez "Initialize repository with a README"
6. Cliquez sur **Create project**

Ensuite, clonez le projet en local :

```
git clone https://gitlab.com/VOTRE-UTILISATEUR/tp7-devsecops.git
cd tp7-devsecops
```

Etape 3 : Preparer le code

Copiez les fichiers du projet-demo dans votre depot :

```
tp7-devsecops/
src/
  app.js
  utils.js
tests/
  utils.test.js
  api.test.js
scripts/
  build.js
  sast-check.js      <-- NOUVEAU
  secret-detection.js <-- NOUVEAU
package.json
.gitlab-ci.yml      <-- NOUVEAU
```

3.1 Copier les fichiers de base

Reprenez les fichiers `src/`, `tests/`, `scripts/build.js` et `package.json` du projet-demo (identiques aux TPs precedents).

3.2 Ajouter les scripts de securite

Creez le dossier `scripts/` s'il n'existe pas, puis ajoutez-y les deux nouveaux fichiers :

- `scripts/sast-check.js` : analyse statique du code

- scripts/secret-detection.js : detection de secrets

Ces fichiers sont fournis dans ce TP (voir les fichiers joints).

Etape 4 : SAST - Analyse statique du code

4.1 Qu'est-ce que le SAST ?

SAST = **S**tatic **A**pplication **S**ecurity **T**esting

C'est l'analyse du code source SANS l'exécuter. On cherche des patterns dangereux directement dans le texte du code.

4.2 Patterns dangereux que notre script detecte

Pattern	Pourquoi c'est dangereux
eval()	Execute du code arbitraire - injection possible
innerHTML	Permet des attaques XSS (Cross-Site Scripting)
document.write	Similaire a innerHTML, risque XSS
exec()	Execute des commandes systeme
new Function()	Cree des fonctions a partir de chaines, comme eval
IP codees en dur	Lie le code a une infrastructure specifique
URLs avec identifiants	Expose des credentials dans le code source

4.3 Le job sast-check dans le pipeline

```
sast-check:
  stage: security
  image: node:18-alpine
  script:
    - echo "=====
    - echo "  SAST - Analyse statique du code"
    - echo "=====
    - node scripts/sast-check.js
  allow_failure: true
```

allow_failure: true : Le pipeline continue meme si des problemes sont trouves. C'est important au debut pour ne pas bloquer toute l'equipe. En production, on passerait a allow_failure: false pour les problemes critiques.

4.4 Comment fonctionne le script sast-check.js

Le script :

1. Lit tous les fichiers .js dans le dossier src/
2. Parcourt chaque ligne de chaque fichier
3. Cherche les patterns dangereux avec des expressions regulieres
4. Affiche les resultats avec le fichier et le numero de ligne
5. Sort avec le code 1 si des problemes critiques sont trouves

Etape 5 : Scan des dependances

5.1 Pourquoi scanner les dependances ?

Votre code peut etre parfait, mais si vous utilisez une librairie avec une faille connue, votre application est vulnérable. npm audit verifie vos dependances contre une base de donnees de vulnerabilites connues.

5.2 Le job dependency-scan

```
dependency-scan:
  stage: security
  image: node:18-alpine
  script:
    - echo "=====
    - echo "  Scan des dependances npm"
    - echo "=====
    - npm ci --cache .npm --prefer-offline
    - echo "--- Resultats npm audit ---"
    - npm audit --audit-level=${SECURITY_LEVEL} || true
    - echo "--- Audit termine ---"
    - npm audit --audit-level=${SECURITY_LEVEL}
  allow_failure: true
```

5.3 Les niveaux d'audit npm

Niveau	Description
low	Toutes les vulnerabilites (meme mineures)
moderate	Moderees et au-dessus
high	Hautes et critiques uniquement
critical	Critiques uniquement

La variable SECURITY_LEVEL permet de configurer ce seuil sans modifier le pipeline.

Etape 6 : Detection de secrets

6.1 Pourquoi detecter les secrets ?

C'est l'une des erreurs les plus courantes et les plus dangereuses :

```
// NE JAMAIS FAIRE CELA !
const API_KEY = "sk-1234567890abcdef";
const DB_PASSWORD = "monsuperpassword";
```

Si ce code est pousse sur un depot (meme prive), ces secrets sont exposes dans l'historique Git POUR TOUJOURS.

6.2 Patterns detectes par notre script

Pattern	Exemple
Mots de passe	password = "secret123"
Cles API	api_key = "sk-abc123..."
Tokens	token = "eyJhbGciOi..."
Cles privees	-----BEGIN RSA PRIVATE KEY-----
Cles AWS	AKIAIOSFODNN7EXAMPLE
Chaines de connexion	mongodb://user:pass@host

6.3 Le job secret-detection

```
secret-detection:
  stage: security
  image: node:18-alpine
  script:
    - echo "=====
    - echo "  Detection de secrets"
    - echo "=====
    - node scripts/secret-detection.js
  allow_failure: true
```

6.4 Bonne pratique : utiliser les variables CI/CD

Au lieu de coder en dur les secrets, utilisez les variables CI/CD de GitLab :

1. Allez dans **Settings > CI/CD > Variables**
2. Ajoutez vos variables (ex: API_KEY, DB_PASSWORD)
3. Cochez **Masked** pour les cacher dans les logs
4. Cochez **Protected** pour les limiter aux branches protegees

Dans le code, utilisez `process.env.API_KEY` au lieu de la valeur en dur.

Etape 7 : Rapport de securite

7.1 Le job security-report

Ce job s'execute a la fin du pipeline et genere un resume de toutes les verifications de securite :

```

security-report:
  stage: report
  image: node:18-alpine
  script:
    - echo "====="
    - echo "  RAPPORT DE SECURITE"
    - echo "====="
    - echo ""
    - echo "--- SAST (Analyse Statique) ---"
    - node scripts/sast-check.js || echo "SAST a trouve des problemes"
    - echo ""
    - echo "--- Dependances ---"
    - npm audit --audit-level=low 2>&1 || echo "Des vulnerabilites ont ete trouvees"
    - echo ""
    - echo "--- Secrets ---"
    - node scripts/secret-detection.js || echo "Des secrets potentiels ont ete trouves"
    - echo ""
    - echo "====="
    - echo "  FIN DU RAPPORT"
    - echo "====="
  allow_failure: true

```

7.2 Lire le rapport

Apres l'execution du pipeline :

1. Allez dans **CI/CD > Pipelines**
2. Cliquez sur le pipeline
3. Cliquez sur le job security-report
4. Lisez les logs pour voir le resume complet

Etape 8 : La philosophie "Shift Left"

8.1 Ordre des stages dans notre pipeline

```
install --> security --> test --> build --> report
```

Remarquez que **security vient AVANT test et build**. C'est le shift left en action :

- Si le code contient des secrets, on le sait AVANT de builder
- Si une dependance est vulnerable, on le sait AVANT de deployer
- Si le code utilise des patterns dangereux, on le sait AVANT les tests

8.2 Progression recommandee

Phase	allow_failure	Comportement
Decouverte	true	Le pipeline continue, on observe
Sensibilisation	true	On commence a corriger les alertes
Enforcement	false	Le pipeline bloque si probleme critique

Dans ce TP, nous sommes en phase de decouverte. En production, vous passeriez progressivement a la phase d'enforcement.

Etape 9 : Variables pour les seuils de securite

9.1 Variables dans le `.gitlab-ci.yml`

```
variables:  
  NODE_VERSION: "18"  
  SECURITY_LEVEL: "moderate"
```

9.2 Surcharger via l'interface GitLab

Vous pouvez changer SECURITY_LEVEL sans modifier le code :

1. **Settings > CI/CD > Variables**
2. Ajoutez SECURITY_LEVEL avec la valeur high ou critical
3. La variable de l'interface GitLab ecrase celle du `.gitlab-ci.yml`

9.3 Surcharger pour un seul pipeline

1. Allez dans **CI/CD > Pipelines > Run pipeline**
2. Ajoutez la variable SECURITY_LEVEL = critical
3. Ce pipeline utilisera critical, les suivants reviendront a la valeur par default

Etape 10 : Lancer le pipeline

10.1 Pousser le code

```
git add .  
git commit -m "feat: ajout du pipeline DevSecOps avec SAST, scan dependances et detection secrets"  
git push origin main
```

10.2 Observer le pipeline

1. Allez dans **CI/CD > Pipelines**
2. Vous devriez voir 5 stages : install, security, test, build, report
3. Les jobs de securite auront un point d'exclamation orange (allow_failure)
4. Cliquez sur chaque job pour lire les resultats

10.3 Points a observer

- Les jobs de securite s'executent en parallele dans le stage security
- Meme si un job de securite echoue, le pipeline continue

- Le rapport final dans security-report resume tout

Tableau recapitulatif

Concept	Description	Job dans le pipeline
SAST	Analyse du code sans l'executer	sast-check
Dependency Scan	Verification des librairies npm	dependency-scan
Secret Detection	Recherche de secrets codes en dur	secret-detection
Shift Left	Securite le plus tot possible	Stage security avant test
allow_failure	Pipeline continue malgre les alertes	Sur tous les jobs securite
Variables	Seuils configurables	SECURITY_LEVEL
Rapport	Synthese de toutes les verifications	security-report

Erreurs courantes

Erreur 1 : "node: command not found" dans les scripts de securite

Cause : L'image Docker ne contient pas Node.js.

Solution : Verifiez que l'image est bien node:18-alpine (pas alpine tout seul).

Erreur 2 : Le script sast-check ne trouve aucun fichier

Cause : Le dossier src/ n'existe pas ou est vide.

Solution : Verifiez que vous avez bien copie les fichiers du projet-demo.

Erreur 3 : npm audit echoue avec "no package-lock.json"

Cause : Le fichier package-lock.json n'est pas present.

Solution : Lancez npm install en local d'abord pour generer le package-lock.json, puis committez-le.

Erreur 4 : YAML invalide avec les " : " dans les echo

Cause : YAML interprete : (espace-deux-points-espace) comme un separateur cle/valeur.

Solution : Entourez toute la ligne avec des guillemets simples :

```
# MAL :  
- echo "Niveau : moderate"  
  
# BIEN :  
- 'echo "Niveau : moderate"'
```

Erreur 5 : Le job security-report echoue

Cause : Les dependances ne sont pas installees dans ce job.

Solution : Le job security-report a besoin du cache avec node_modules/. Verifiez la configuration du cache.

Erreur 6 : "ENOENT: no such file or directory" dans secret-detection

Cause : Le script essaie de lire un fichier qui n'existe pas.

Solution : Verifiez que tous les fichiers du projet sont bien commites et pousses.

Pour aller plus loin

- **GitLab SAST officiel** : GitLab propose des templates SAST integres (include: - template: Security/SAST.gitlab-ci.yml)
- **Snyk** : Outil professionnel de scan de dependances, s'integre a GitLab
- **TruffleHog** : Outil professionnel de detection de secrets dans l'historique Git
- **OWASP Top 10** : Les 10 risques de securite les plus courants pour les applications web

Bon TP ! La securite n'est pas une option, c'est une habitude.